

Tarea 3: Diseño Avanzado de Sistemas Digitales

1. Descripción General

La genómica se enfoca en el estudio integral del ADN de los organismos. Dada la masividad de estos datos, su análisis computacional suele apoyarse en el uso de k -mers: subcadenas de longitud fija k extraídas de secuencias genéticas más largas mediante el método de ventana deslizante.

Al funcionar como las piezas de un rompecabezas, los k -mers permiten a la bioinformática ensamblar genomas, identificar especies y comparar secuencias de manera eficiente, transformando datos biológicos complejos en unidades manejables para los algoritmos.

En el ADN, las bases nitrogenadas (Adenina, Timina, Citosina y Guanina) se agrupan en estas secuencias de longitud k . Para evitar redundancias debido a la naturaleza de la doble hélice, se utiliza el k -mer canónico: la representación estandarizada que corresponde a la cadena lexicográficamente menor entre el k -mer original y su reverso complementario. El objetivo de esta tarea es diseñar e implementar un sistema de cómputo heterogéneo CPU-FPGA que optimice el procesamiento genómico. El sistema deberá comparar los k -mers de dos genomas distintos para determinar cuántos de ellos son idénticos y coinciden exactamente en la misma posición, fijando el valor de $k = 31$.

2. Entradas, Salidas e Interfases del Sistema

Para la evaluación del sistema, se proveerá una tarjeta SD con dos archivos que contienen dos genomas a comparar. Cada genoma estará representado como una lista de 31-mers canónicos, dispuestos a razón de uno por línea.

El procesamiento y la comunicación entre el sistema de procesamiento (PS) y la lógica programable (PL) se estructurarán bajo los siguientes lineamientos:

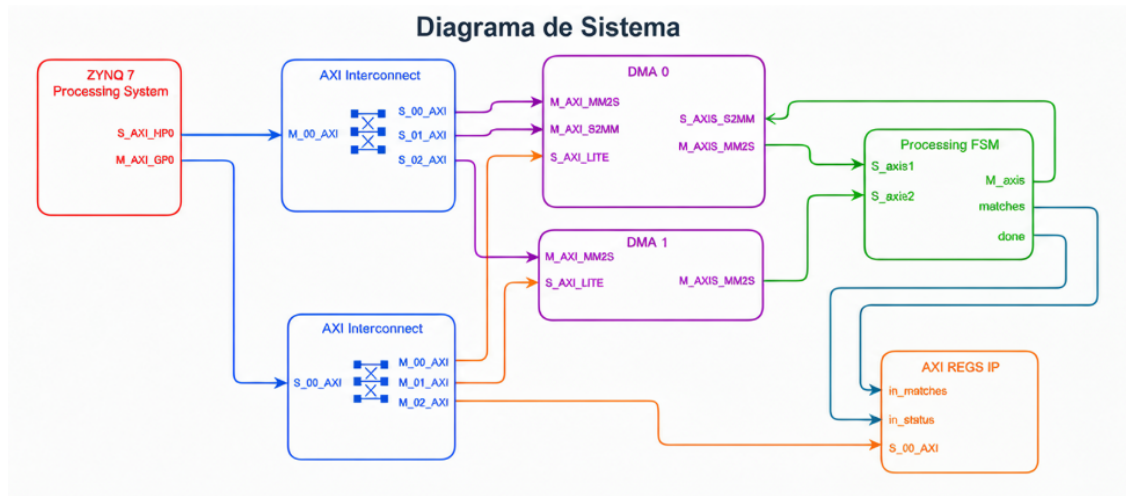
- **Transferencia de Datos (DMA):** Los k -mers de ambos genomas deberán ser transferidos hacia la PL mediante dos módulos de acceso directo a memoria (DMA) con un ancho de banda de 32 bits. Dado el tamaño de cada componente, cada 31-mer requerirá exactamente **8 palabras de 32 bits** para completarse secuencialmente.
- **Lógica de Comparación y Stream de Salida:** Se deberá diseñar e implementar una máquina de estados finitos (FSM) encargada de sincronizar ambos flujos de datos entrantes y realizar la comparación término a término (k -mer a k -mer). Como resultado, la FSM generará un flujo de salida continuo (*stream*) compuesto por palabras de 32 bits. Cada bit de estas palabras representará el resultado de una comparación individual (un 1 lógico si los k -mers coinciden, o un 0 lógico en caso contrario). Por lo tanto, cada palabra de salida empaquetará el resultado de 32 comparaciones consecutivas.
- **Registro de Control y Estado (IP AXI REGS):** El sistema integrará un bloque de propiedad intelectual (IP) preconfigurado denominado AXI REGS, el cual expone dos registros de entrada que son legibles desde el software en el PS:

- **in_matches** (32 bits): Registro acumulador que almacena la cantidad total de coincidencias (k -mers idénticos en la misma posición) detectadas durante la ejecución.
- **in_status** (32 bits): Registro de control cuyo bit menos significativo (*LSB*) actúa como una bandera (*flag*) para indicar la finalización del procesamiento de ambos genomas.

Usted debe conectar las salidas de la lógica desarrollada a las entradas de este bloque IP.

3. Diagrama y elementos del sistema

La Figura 1 muestra los elementos del sistema y como se conectan entre ellos.



3.1. Zynq-7 Processing System (PS)

El sistema de procesamiento (PS) actuará como el elemento coordinador central del sistema heterogéneo, gestionando el flujo de datos, la configuración del hardware y el análisis estadístico posterior. Las responsabilidades del procesador abarcan las siguientes tareas:

- **Gestión de Almacenamiento:** Lectura de los archivos de entrada (genomas) desde la tarjeta SD y posterior escritura del histograma de resultado en el mismo medio de almacenamiento.
- **Procesamiento en Software (Baseline):** Calcular mediante software el número de coincidencias de k -mers entre ambos genomas, sirviendo como una implementación de referencia para validar la correctitud de la lógica en hardware.
- **Control de Periféricos de Memoria (DMA):** Configurar, inicializar y gestionar el ciclo de vida (incluyendo operaciones de reinicio o *reset*) de los módulos de acceso directo a memoria (DMA).
- **Orquestación del Flujo de Datos (Tx/Rx):**
 - Transferir los flujos de datos genómicos leídos desde la SD hacia la lógica programable (PL) a través de los canales DMA de transmisión.
 - Recibir el flujo de salida (*stream*) generado por la PL mediante un canal DMA de recepción.

- **Monitoreo y Sincronización:** Monitorear el registro `in_status` para determinar con precisión el instante en que la PL finaliza el procesamiento.
- **Lectura de Métricas:** Acceder al registro `in_matches` del bloque `AXI_REGS` para recuperar el contador total de coincidencias calculadas por el hardware.
- **Análisis Post-procesamiento:** Procesar el *stream* de bits recibido de la PL para generar un histograma que represente la frecuencia de ráfagas de coincidencias consecutivas (por ejemplo, cuantificar cuántas veces se presentaron secuencias de exactamente N k -mers idénticos sucesivos).

Dejen la frecuencia del `FCLK_CLK0` en 125 MHz.

3.2. AXI Interconnects

Estos módulos actúan como la infraestructura de conmutación e interconexión interna del sistema. Su función principal es canalizar el tráfico entre las interfaces de propósito general (`M_AXI_GP`) y de alto rendimiento (`S_AXI_HP`) del PS, los bloques DMA y los registros de control.

El bloque *AXI Interconnect* gestiona de forma transparente las siguientes subtarefas críticas de hardware:

- **Conversión de Protocolos:** Conversión mutua entre las distintas variantes de la familia AXI. Esto incluye la degradación de comandos desde el bus maestro del PS (*AXI3/AXI4-Full*) hacia el protocolo simplificado *AXI4-Lite* requerido por los registros de control del DMA y del bloque `AXI_REGS`, así como la adaptación de las ráfagas *AXI4-Full* provenientes de los canales de datos hacia la memoria.
- **Conversión de Ancho de Datos (*Data Width Conversion*):** Adaptación de los tamaños de palabra entre maestros y esclavos con buses de distinto ancho (por ejemplo, adaptando accesos de 32 bits a buses de 64 bits).
- **Sincronización de Dominios de Reloj (*Clock Domain Crossing*):** Manejo seguro e inserción de estructuras de sincronización cuando la lógica programable (PL), las interfaces de memoria y el procesador operan a frecuencias de reloj distintas.
- **Arbitraje de Tráfico:** Gestión de prioridades y resolución de conflictos de acceso cuando múltiples maestros intentan acceder concurrentemente a un mismo puerto esclavo de memoria.

3.3. AXI Direct Memory Access (DMA)

Los módulos AXI DMA son los encargados de realizar la transferencia masiva de datos mediante el acceso directo a la memoria DDR, liberando al procesador de esta carga. Su función principal en este sistema es generar los flujos de datos entrantes (*streams*) hacia la lógica programable y retornar los resultados procesados hacia la memoria del sistema.

La arquitectura de canales de transferencia para este proyecto se distribuye de la siguiente manera:

- **AXI DMA 0:** Configurado de forma bidireccional, disponiendo de un canal de lectura (*MM2S* - *Memory Map to Stream*) para enviar el primer flujo genómico y un canal de escritura (*S2MM* - *Stream to Memory Map*) para almacenar el flujo de salida generado por la FSM.
- **AXI DMA 1:** Configurado de forma unidireccional, disponiendo únicamente de un canal de lectura (*MM2S*) para transferir el segundo flujo genómico.

3.3.1. Protocolos e Interfaces de Comunicación

Para interactuar con los distintos elementos del sistema heterogéneo, cada bloque DMA implementa tres variantes del protocolo AXI:

1. **AXI4-Full (Memory Mapped):** Interfaz utilizada para realizar las transacciones de lectura y escritura por ráfagas de alta velocidad directamente en la memoria DDR del PS.
2. **AXI4-Stream:** Interfaz punto a punto de baja latencia utilizada para la comunicación de datos secuenciales con la máquina de estados (FSM) diseñada por el usuario.
3. **AXI4-Lite:** Interfaz esclava de registros mediante la cual el PS (software) configura, inicializa y monitorea el estado del periférico.

Configuraciones Importantes del DMA

Para el correcto funcionamiento del sistema, es importante modificar los siguientes parámetros pre-determinados en el asistente de configuración de ambos bloques DMA:

- **Deshabilitar el *Scatter Gather Engine*.**
- **Width of Buffer Length Register:** Configurar este registro exactamente a **23 bits** para permitir transmitir los genomas en una sola instrucción.

3.4. Processing FSM

Este módulo constituye el núcleo del hardware que deberá ser diseñado e implementado por usted. Su función principal es recibir los streams de ambos genomas, alinearlos, realizar la comparación de los k -mers y empaquetar los resultados en el flujo de salida.

Al diseñar la lógica y la máquina de estados finitos (FSM), se deben cumplir los siguientes criterios de diseño:

- **Sincronización de *Streams*:** Debido a que los datos provienen de dos canales DMA independientes, no se puede garantizar que los k -mers de ambos genomas lleguen simultáneamente. La FSM debe implementar una barrera de sincronización basada en el protocolo *AXI-Stream*:
 - Se debe esperar a que la señal **TVALID** de ambos canales de entrada esté en alto antes de proceder con la comparación.
 - No se deben consumir ni descartar los datos (manteniendo la señal **TREADY** en bajo) hasta que la comparación del par de k -mers actual se haya ejecutado.
- **Gestión de Fin de Flujo (TLAST) y Genomas Asimétricos:** Se debe verificar la señal **TLAST** de cada canal de entrada para identificar el término de los datos de cada genoma. En caso de que los genomas tengan tamaños distintos, la comparación activa debe realizarse **únicamente hasta que uno de los dos genomas se agote** (es decir, tan pronto como se detecte un pulso en la señal **TLAST** de cualquiera de los dos *streams*). Una vez que esto ocurre, se activa el registro que indica que el procesamiento terminó.
- **Gestión y Polaridad del *Reset*:** Debe diseñar una lógica de reinicio que devuelva la FSM y los contadores internos a un estado inicial conocido. Se debe verificar con especial cuidado la **polaridad de la señal de reset** proveniente del bloque del sistema (por ejemplo, el bloque *Processor System Reset* de Xilinx suele entregar por defecto señales de reset activo en bajo).

- **Generación del *Stream* de Salida:** La FSM debe agrupar los resultados de las comparaciones individuales en palabras de 32 bits, controlando correctamente los (*handshakes*) del canal de transmisión del DMA 0 para evitar la pérdida de datos.

3.5. AXI REGS IP

Este bloque de propiedad intelectual (IP) preconfigurado actúa como el puente de comunicación directa para el intercambio de métricas y banderas de control entre la lógica programable (PL) y el sistema de procesamiento (PS). El módulo implementa una interfaz esclava *AXI4-Lite* que mapea en memoria dos registros internos mapeados para su lectura desde el software en el procesador:

- **in_matches:** Registro de 32 bits que expone de manera continua el conteo acumulado de coincidencias detectadas por la FSM.
- **in_status:** Registro de 32 bits cuyo bit menos significativo (*LSB*) actúa como la señal de finalización (*in_done*), indicando al software que el procesamiento de los genomas ha concluido.

El módulo se entregará como un componente empaquetado listo para ser usado en el diagrama de bloques.

4. Anexos y Comentarios Relevantes

4.1. Lectura de Archivos desde la Tarjeta SD

Para las pruebas del sistema, podrán retirar una tarjeta MicroSD en el laboratorio o utilizar una propia. En ella deberán almacenar los archivos de datos genómicos *31mers_5.txt* y *31mers_6.txt*, los cuales estarán disponibles en la plataforma Teams.

Se subirá un archivo con el código de referencia necesario para gestionar el sistema de archivos (FATFS) y realizar la lectura desde la SD en el entorno embebido. Asimismo, se dispondrá de un video explicativo detallando la integración de este código y consideraciones críticas de configuración; es de carácter obligatorio revisar este material antes de comenzar la implementación.

4.2. Transferencias por DMA

Las plantillas de código para el control, inicialización y gestión de transferencias por DMA se encuentran publicadas en Teams. Estos ejemplos demuestran cómo transmitir y recibir flujos de datos (*streams*) y deben ser adaptados para cumplir con los requerimientos específicos de esta tarea.

Los identificadores de hardware (*Device IDs*) requeridos para inicializar las instancias de los bloques DMA en el controlador de software se obtienen del archivo de configuración de la plataforma (*xparameters.h*), tal como se ilustra en el Código 1:

Listing 1: Definición de identificadores de hardware para los bloques DMA.

```
#include "xparameters.h"

// Identificadores de dispositivo para los bloques DMA
#define DMA_0_ID          XPAR_AXI_DMA_0_DEVICE_ID
#define DMA_1_ID          XPAR_AXI_DMA_1_DEVICE_ID
```

4.3. Lectura de Registros del IP AXI

Para interactuar con el bloque AXI REGS desde el software del PS, se debe mapear la dirección base asignada por Vivado al periférico utilizando la constante generada en `xparameters.h` (Código 2):

Listing 2: Mapeo de la dirección base del IP de registros AXI.

```
#define MY_IP_BASE          XPAR_AXIREGS_0_S00_AXI_BASEADDR
```

La lectura de los registros internos se realiza mediante accesos directos a memoria de 32 bits utilizando las funciones nativas de Xilinx. Los registros están espaciados por un (*offset*) de 4 bytes debido al direccionamiento por palabra, donde el registro 0 (`in_matches`) posee un *offset* de 0x0 y el registro 1 (`in_status`) un *offset* de 0x4 (Código 3):

Listing 3: Lectura de los registros de estado y coincidencias mediante software.

```
#include "xil_io.h" // Requerido para las funciones Xil_In32

// Lectura de registros utilizando offsets de memoria
u32 status_reg = Xil_In32(MY_IP_BASE + 4); // Registro 1: in_status
u32 matches    = Xil_In32(MY_IP_BASE + 0); // Registro 0: in_matches
```

4.4. Troubleshooting y Depuración

Al igual que se revisó en las sesiones prácticas, el método fundamental para depurar el software y verificar el estado del sistema es el despliegue de mensajes en la consola serial (UART). Para ello, se debe utilizar la función optimizada `xil_printf` en lugar del `printf` estándar de C.

Por otra parte, el entorno de desarrollo Vitis suele presentar problemas de sincronización al actualizar la especificación del hardware tras realizar modificaciones en el diagrama de bloques de Vivado. Para evitar errores de compilación por inconsistencia de componentes, se recomienda seguir estrictamente el siguiente procedimiento secuencial cada vez que se exporte un nuevo archivo `.xsa`:

1. **Limpieza del Proyecto:** En la vista *Vitis Explorer* (panel izquierdo), haga clic derecho sobre el proyecto de plataforma ([Platform]) y seleccione **Clean Project**.
2. **Actualización de la Especificación de Hardware:** Haga clic derecho nuevamente sobre el proyecto de plataforma y seleccione **Update Hardware Specification**.
3. **Vinculación del nuevo archivo:** Busque (*Browse*) y seleccione el archivo `.xsa` recién exportado desde Vivado y haga clic en **OK**.
4. **Regeneración Manual del BSP (¡No compile aún!):**
 - Diríjase al panel *Assistant* (por lo general, ubicado en la esquina inferior izquierda).
 - Haga clic derecho sobre la plataforma y seleccione **Open Platform Editor**.
 - En el árbol de navegación del editor, despliegue y haga clic sobre el dominio del procesador: `standalone_ps7_cortexa9_0`.
 - Seleccione la opción **Board Support Package** y fuerce la reconstrucción completa de las librerías haciendo clic en el botón **Reset BSP Sources**.
5. **Compilación Final:** Una vez que el entorno reconozca los cambios del hardware, compile su proyecto de aplicación en modo *Release*. Para asegurar un binario limpio, realice primero un **Clean** de la aplicación y posteriormente ejecute el **Build**.

5. Informe y Entregables

La fecha límite para la entrega de este proyecto es el **miércoles 1 de julio**. La actividad podrá realizarse en grupos de **2 o 3 integrantes**.

Se deberá adjuntar un informe técnico formal en formato PDF a través de la plataforma establecida, el cual debe documentar detalladamente el diseño y los resultados experimentales. El contenido del informe y los archivos entregables deben cumplir con la siguiente estructura:

5.1. Documentación del Diseño y Código

- **Diagrama de Bloques General**
- **Módulo de Comparación RTL:** Descripción detallada de la máquina de estados finitos (FSM) diseñada, incluyendo diagramas de transición de estados y el código fuente en SystemVerilog/Verilog debidamente comentado.
- **Software de Aplicación:** Código fuente en C desarrollado en Vitis para la inicialización de periféricos (DMA, UART, AXI REGS), la lectura de la tarjeta SD, el cálculo del modelo de referencia (*Golden Model*) y el post-procesamiento del histograma.

5.2. Resultados Experimentales y Validación

- **Validación de Coincidencias:** Captura de pantalla o transcripción de la consola serial (UART) donde se evidencie y compare el número total de k -mers coincidentes calculado por software frente al obtenido por hardware. Ambos valores deben ser idénticos para validar el diseño.
- **Análisis Estadístico (Histograma):** Inclusión del archivo de texto generado en la tarjeta SD con los datos del histograma de secuencias consecutivas, acompañado de un gráfico (por ejemplo, gráfico de barras) que represente visualmente dicha distribución de frecuencias.
- **Métrica de Rendimiento (Tiempos de Ejecución):** Cuadro comparativo con las mediciones de tiempo de ejecución empleadas por la CPU (software puro) y el sistema acelerado (CPU-FPGA).

5.3. Análisis Crítico y Propuestas de Mejora

- **Evaluación del Desempeño:** Discusión y comentarios analíticos sobre el factor de aceleración (*speedup*) obtenido en el hardware en comparación con la ejecución secuencial en el procesador.
- **Propuestas de Optimización:** Presentar un conjunto de ideas técnicas justificadas para mejorar el rendimiento del sistema en la FPGA.

Referencias

- [1] AMD Xilinx. *Zynq-7000 SoC Technical Reference Manual (UG585)*. Disponible en el sitio web de AMD. (*Esencial para entender la arquitectura del PS, los puertos M_AXI_GP y S_AXI_HP, y la gestión de relojes FCLK_CLK0*).

- [2] AMD Xilinx. *AXI Direct Memory Access LogiCORE IP Product Guide (PG021)*. Disponible en el sitio web de AMD. (*Documentación crítica para entender los registros de configuración del DMA, el funcionamiento de las transferencias en modo Simple DMA y el significado físico de los pines MM2S y S2MM*).
- [3] ARM. *AMBA AXI and ACE Protocol Specification (AXI3, AXI4, and AXI4-Lite)*. (*Referencia base para el diseño de la FSM, detallando las reglas del protocolo de handshake entre las señales TVALID y TREADY, así como el uso de TLAST*).
- [4] ChaN. *FatFs - Generic FAT File System Module*. http://elm-chan.org/fsw/ff/00index_e.html (*Documentación oficial de la librería FatFs utilizada por el BSP de Xilinx para abrir, leer y escribir archivos en la tarjeta MicroSD desde C/C++*).